

Data Science and AI Lab Project

GROUP I

RAG-BASED LLM CODE REVIEW AGENT

MILESTONE 3



OBJECTIVES

- Design architecture for Code Review Assistant
- Process PR diffs → structured outputs
- Use the Retrieval-Augmented (RAG) approach
- Combine retrieval + lightweight generation
- Ensure an explainable & modular system
- Build and validate working prototype

DATASET ORGANIZATION & COMPONENTS

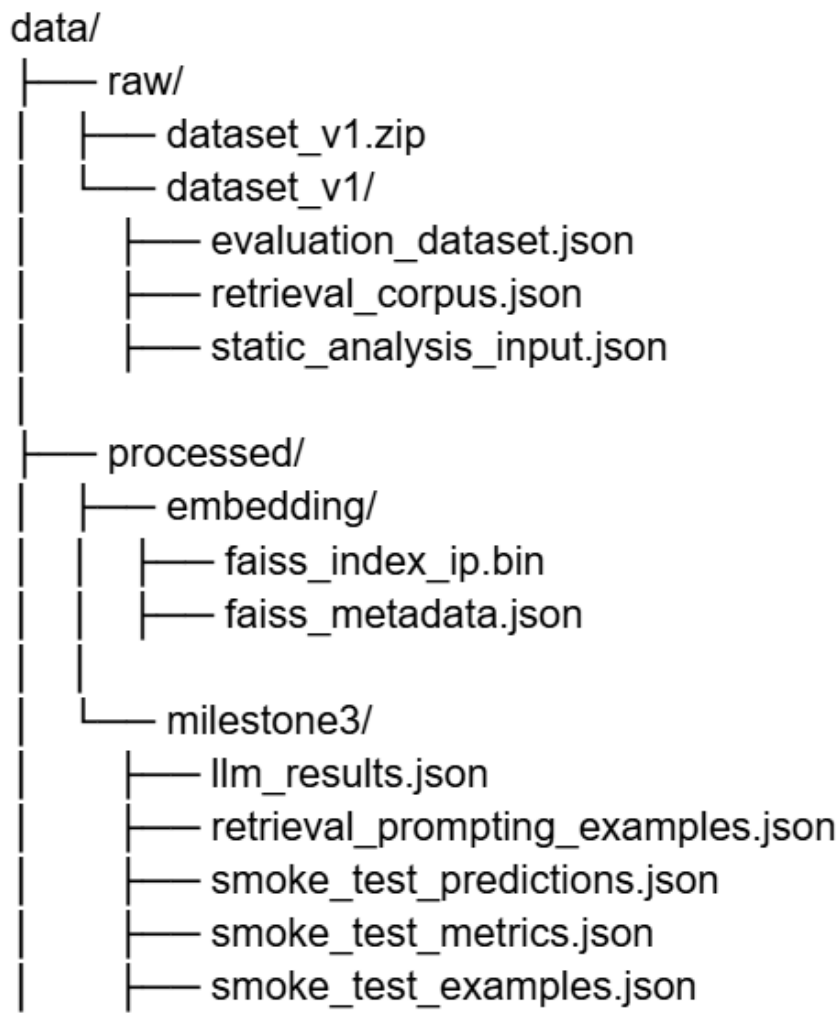


01 Data Structure

Two layers: Raw Data & Processed Data
Ensures clear separation & modular design

02 Directory Overview

- raw/ → original datasets
- Evaluation dataset
 - Retrieval corpus
 - Static analysis input
- processed/ → embeddings & outputs
- FAISS index
 - Milestone 3 results



03 Core Datasets

- **Evaluation Dataset**
PR diffs + labeled review comments
- **Evaluation Dataset**
Coding guidelines (PEP8, linters, reviews)
- **Evaluation Dataset**
Rule-based baseline comparison

DATA FLOW & DESIGN PRINCIPLES



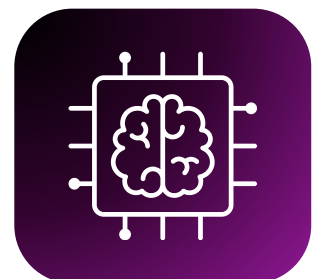
Pipeline Flow

- PR Diff → Preprocessing
- → Embedding Generation
- → FAISS Retrieval (Top-K)
- → Context + Prompt Creation
- → LLM Inference
- → Output: Violations + Review Comments



Generated Outputs

- Predictions & evaluation metrics
- Retrieved context + prompts
- LLM-generated results



Design Principles

- Separation of raw & processed data
- Scalable & reproducible pipeline
- Optimized for RAG architecture
- Fast semantic search using FAISS



DATA PREPROCESSING PIPELINE



01 Purpose

- Convert raw PR diffs → structured inputs
- Enable semantic retrieval + LLM inference

02 Evaluation Dataset Processing

- Extract relevant diff chunks
- Clean & normalize code (remove symbols)
- Build context: file path + diff + metadata
- Optional tokenization for hints

03 Retrieval Corpus Processing

- Clean & normalize documents
- Split into 200–400 token chunks
- Add metadata: category, source, chunk ID

04 Embedding Generation

- Model: BGE-large (SentenceTransformers)
- Convert queries & corpus → vectors
- Normalize embeddings for similarity search

RETRIEVAL, PROMPTING & OUTPUT



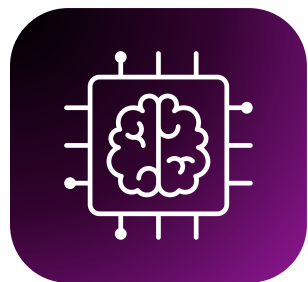
FAISS Indexing

- Store embeddings for fast retrieval
- Top-K similarity search (K=5)
- Returns relevant guideline chunks



Retrieval & Context Formation

- Query → embedding → FAISS search
- Retrieve top relevant rules
- Build structured context (text + metadata)



Prompt Construction

- Combine:
 - PR diff
 - Retrieved guidelines
 - Instructions
- Enforces structured JSON output

Final Output: Generates violation category and context-aware review comment based on retrieved guidelines.



MODEL ARCHITECTURE OVERVIEW

Approach

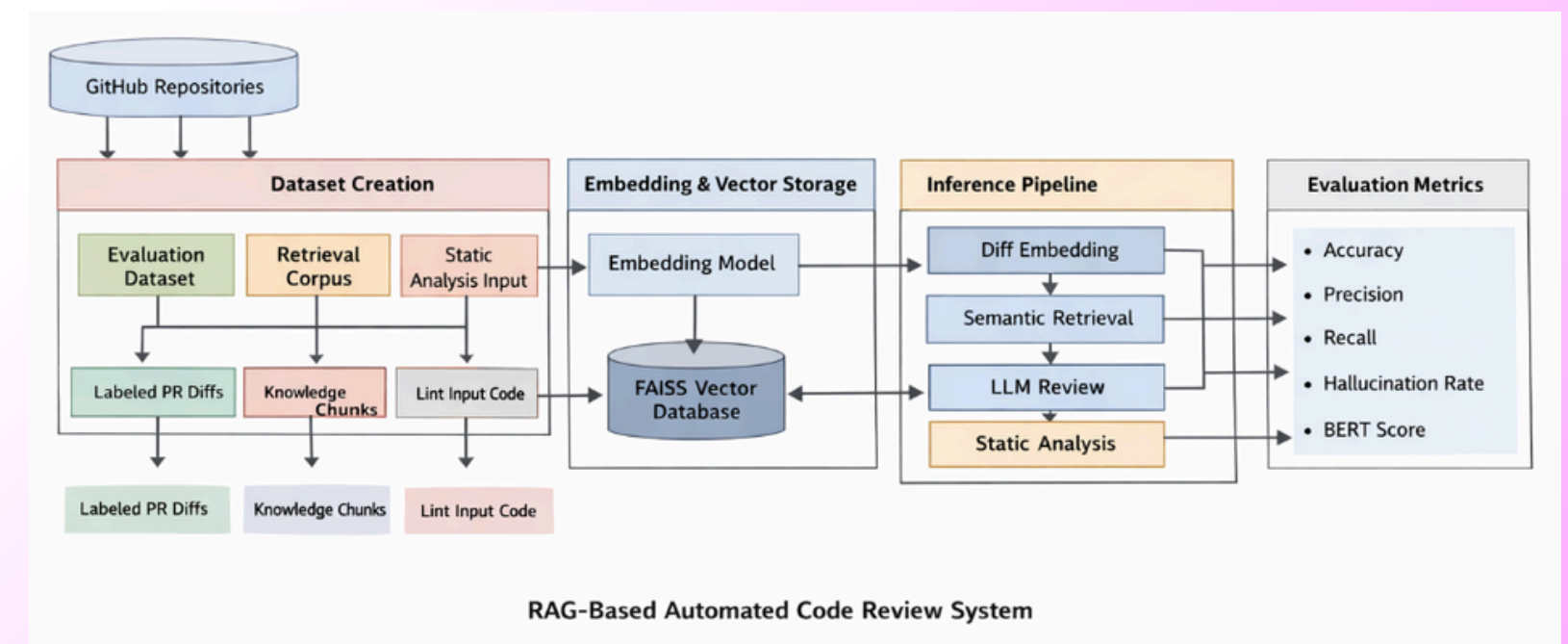
- Retrieval-Augmented Generation (RAG-based system)
- Combines semantic retrieval + LLM reasoning

Input Unit

- Single violation instance
- Includes: diff chunk, file path, line number, PR metadata

Pipeline Stages

- Preprocessing & query construction
- Embedding generation
- FAISS-based retrieval
- Prompting + LLM generation
- Grounding & validation



Key Goal : Generate accurate, explainable review comments

CORE COMPONENTS



01 **Diff Processing**

- Extract diff chunks based on line numbers
- Clean diff (remove symbols, preserve structure)
- Segment into 20–50 line chunks

02 **Embedding & Retrieval**

- Model: BAAI/bge-large-en-v1.5
- SentenceTransformers
- Dense vector embeddings (normalized)

03 **Prompt Construction**

- Combines:
 - PR diff
 - Retrieved guidelines
 - Metadata + instructions
- Ensures structured JSON output

04 **LLM Generation**

- Outputs:
 - Violation category
 - Grounded review comment

VALIDATION, OUTPUT & DESIGN



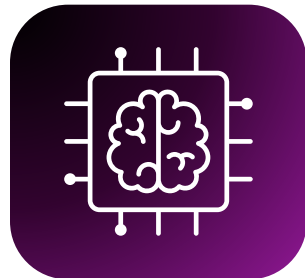
Grounding & Validation

- Check alignment with retrieved evidence
- Detect hallucinations
- Measure grounding quality



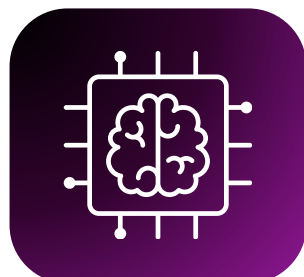
Static Analysis (Baseline)

- Tools: Flake8, Pylint
- Compare:
 - Rule-based vs LLM vs RAG



Final Output Format

- Category + review comment
- Includes referenced guideline chunks



Limitations

- Limited evaluation dataset size
- Limited retrieval corpus coverage
- Imbalance across violation categories
- Depends on prompt design and LLM behavior

The final output is structured as:

```
{
  "category": "<violation_type>",
  "grounded_comment": "<review comment>",
  "cited_chunk_ids": ["chunk_id_1", ...]
}
```



INPUT & EMBEDDING REPRESENTATION



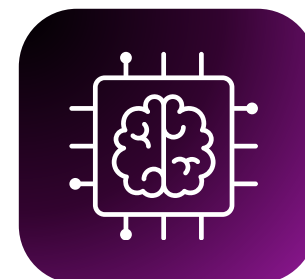
Input

- Diff chunk
- File path
- Repo metadata (repo, PR ID, line no.)



Retrieval Corpus Representation

```
{  
  "chunk_id": "...",  
  "text": "...",  
  "category": "...",  
  "source_type": "..."  
}
```



Embedding Representation

- Model: BAAI/bge-large-en-v1.5
- Vector: 1024-dim, L2-normalized



RETRIEVAL & PROMPT CONSTRUCTION

FAISS Retrieval

- Stores corpus embeddings
- Input: query vector (1×1024)
- Output:
 - Top-K results ($k=5$)
 - Similarity scores



Prompt Structure (LLM Input)

[Instruction]

Context:

- repo
- file_path
- line_number
- category_hint (optional)

Diff:

<diff_chunk>

Retrieved Evidence:

- chunk_id, category, source, score, text

Output Format:

JSON with fields:

- category
- grounded_comment
- cited_chunk_ids

END-END PIPELINE



- 01 **Input Processing**
Generate structured query from PR diff (file path + diff chunk)
- 02 **Embedding (BGE-large)**
Convert query into 1024-dimensional L2-normalized vector
- 03 **FAISS Retrieval (Top-K)**
Retrieve Top-K = 5 most similar guideline chunks
- 04 **Prompt Construction**
Combine instruction, metadata, diff, and retrieved evidence
- 05 **LLM Inference**
Generate code review using structured prompt
- 06 **Structured Output (JSON) and Evaluation**
Produce category, grounded_comment, and cited_chunk_ids and Evaluate.

OUTPUT REPRESENTATION

Stored Files

- smoke_test_predictions.json
-full predictions
- smoke_test_metrics.json
-Evaluation metrics
- smoke_test_examples.json
- Sample outputs



Sample Output

```
{  
  "pr_id": "PR_4682",  
  "repo": "pallets/flask",  
  "file_path": "src/flask/globals.py",  
  "line_number": 73,  
  "gold_category": "naming_convention",  
  "groq_predicted_category": "unused_import",  
  "groq_grounded_comment": "Remove the unused  
imports of `functools.partial` and  
`werkzeug.local.LocalStack` to keep the module  
clean.",  
  "groq_cited_chunks": ["chunk_0600",  
"chunk_0571"]  
}
```

EVALUATION SETUP



One-vs-rest classification per violation category

Prediction is correct if:

- Line number matches ($\pm$ tolerance)
- Category matches ground truth

Outputs compared against ground truth:

- True Positive (TP): Correct prediction of a violation category
- False Positive (FP): Incorrect prediction of a category
- False Negative (FN): Missed violation
- True Negative (TN): Correct rejection of non-category



Grounding Evaluation

Based on retrieved knowledge

Grounded if:

- References guideline
- Cites standard
- Aligns with retrieved docs



Latency Setup

Measured per instance

Same hardware for all systems

- RAG:
 - Retrieval + Generation
- LLM:
 - Generation only
- Static tools:
 - Analysis runtime

EVALUATION METRICS



Classification Metrics

- Precision = $TP / (TP + FP)$
- Recall = $TP / (TP + FN)$
- F1 Score = balance of precision & recall
- Macro F1 → average across categories



Grounding Rate

Grounding Rate = grounded / total



Semantic Alignment

Compared with human reviews

Metric: BERTScore (F1)

Threshold: > 0.85



Hallucination Rate

Incorrect or unsupported outputs

Hallucination Rate = hallucinated / total



Latency Metrics

Mean latency → average performance

P95 latency → worst-case performance